Aarhus University

Project number - 2026F25

Parallax - Technology Analysis

Students:
Thea Teresa Lindgaard (202204350)
Niels Jakob (202105879)
Kasper Stecher (202100451)
Kasper Bentsen (202103249)

Counsellor:
Lukas Esterle

Aarhus, May 29th 2026

Contents

1. Choice of Game Engine

Choosing the game engine for the project comes down to setting the demands and need for the project and finding the best overall suited tool.

First a short demands list is needed to set the requirements for the project. Some of the major demands are that the tool supports 3D development and online game play. These demands narrowed the list to the following three tools:

1) Unity
2) Unreal Engine
3) Godot

Setting up demands for the development and thereby judging each of the tool.

- Learning curve and prior experience (High Importance)
- 3D support (High Importance)
- Online Documentation (Medium Importance)
- Performance and Flexibility (Low Importance)
- Online multiplayer support (Medium Importance)

In the following sections, each game engine will rated according to each requirement. In addition, each requirement will be briefly described, and weighted according to importance.

1.1. Learning curve and prior experience

When choosing a game engine the learning curve and prior experience has high importance as it determines how fast and seamless the development can begin. Using a new game engine with no experience will lead to a slower start as each student would have to get settled into a new environment. Thus this requirement is of **high importance**.

In this group every member has prior experience with game development in **Unity** through different courses and educations, placing **Unity** at the top with regards to learning curve and prior experience.

Only one group member has limited experience with Unreal Engine. Additionally, Unreal Engine is considered to have a steeper learning curve compared to Unity **[1]**.

No member of the group has experience in Godot. However, Godot's learning curve is not as steep as Unreal, and is in many aspects comparable to Unity **[2]**.

1.2. 3D support

For engine choice, the 3D support offered natively by the engine is of **high priority**. The vision of the game is a 3D cooperative puzzle game, as a 3D environment offers much more flexibility in puzzle and world design. Since the development time is limited, native 3D support from the engine is important **[3]**.

While researching game engines, it's clear that Unreal has a bit more effective 3D support, since that is what the engine is primarily intended for. Godot and Unity roughly provide the same 3D features, but per the two year old article Unity Vs Godot**[2]**, Godots 3D development comes with worse rendering capabilities and some manual labor for effective camera management. The content in the article may be somewhat outdated, but since the scale of the game is fairly small, Unity and Godot score the same.

1.3. Online Documentation

Online documentation and resources for an engine can be relevant, especially if the group is unfamiliar with the chosen engine. This requirement is considered to be of **medium importance**, as familiarity and learning curve of the engine will affect how reliant we will be on documentation.

All engines have documentation and tutorials available online. However, Unity and Unreal, being more mature technologies, have more documentation available. Godot has a smaller community, and thus less support **[4]**.

1.4. Flexibility

Because the project may undergo small pivots as the design matures, flexibility is an important technical requirement. The current target is a fully 3D game, but during brainstorms idea such as 2D mini games have already been mentioned, while these are not part of the initial game scope, the engine should support mixing rendering styles, this favors game engines with workflows and tooling support for 2D such as Unity and Godot more, whereas Unreal is much more geared towards 3D game play.

1.5. Performance

Unreal Engine requires a lot from your working machine, which may be an obstacle for indie developers and smaller projects. Extra optimization will require more efforts, and could add a major time consumption. However for larger projects AAA titles and such, the extra optimization can result in an overall better performance.**[4]**

Unity offers performance that scales well across different hardware levels. When it comes to resource-demanding and graphically intensive games, Unity tends to have some occasional hiccups, requiring you to spend more time and effort optimizing the project. But for indie game development this should not become an issue.**[4]**

In comparison to Unreal Engine and Unity, Godot lacks the vast collection of tools, plugins and additional third-party integrations, meaning you have to integrate these things by yourself, leading to it requiring a big amount of experience, time and effort.**[4]**

1.6. Online Multiplayer Support

Since the game concept is highly reliant on online play, choosing an engine with good online multiplayer support is preferable, as it would make it easier to implement. However, if the engine does not support online play out of the box, it *is* possible to manually implement such a feature. Therefore, this requirement is rated as **medium importance**.

Unity supports a package called **NetCode for GameObjects**. This package makes it possible to create a host and connect a client, and automatically sync player position, rotation and scale, with only some simple set-up required in the Unity Editor **[5]**.

Godot does not have third party packages that support online multiplayer in the same way that Unity and Unreal Engine does. That means that Godot support online multiplayer in the "old-school" way, where an IP-address is used to connect to the host. This makes it more complex compared to the other candidates **[6], [7]**.

Unreal engine uses the client-server architecture for networked multiplayer games. Epic games provides comprehensive documentation and tutorial. Developing game play for a multiplayer game requires implementing **Replication** in the game's **Actors**, it is also needed to design functionality specific to the server, which acts as the host for the game session **[8]**.

1.7. Conclusion

Each engine was ranked based on the requirements from the analysis. In **Figure 1** is an overview of how the engines scored, and it shows that Unity scored the highest. Therefore, Unity was chosen as the engine for this project.

	Unreal	**Godot**	**Unity**
Prior Experience	☆☆	☆☆	☆☆☆☆☆
3D Support	☆☆☆☆☆	☆☆☆☆	☆☆☆☆
Online Resources	☆☆☆☆	☆☆☆	☆☆☆☆
Performance	☆☆☆☆☆	☆☆☆☆	☆☆☆☆
Flexibility	☆☆☆	☆☆☆☆	☆☆☆☆☆
Online multiplayer	☆☆☆☆	☆☆	☆☆☆☆
Average Score	**3.83**☆	**3.17**☆	**4.33**☆

Figure 1: An overview of how each engine from the analysis has scored for each requirement. Unity scored the most points, and is the engine chosen for this project.

Bibliography

[1] Gamosophy, “Unity vs Unreal Engine: A Comprehensive Comparison for 2025.” [Online]. Available: https://gamosophy.com/unity-vs-unreal-engine-comparison-2025/#:~:text=Unity's%20ease%20of%20deployment%20makes,%2Dplatform%20titles%20meshy.ai.

[2] M. Jayasri, “Unity vs. Godot: A Game Developer's Guide .” [Online]. Available: https://dev.to/manasajayasri/unity-vs-godot-a-game-developers-guide-2a6o

[3] M. Phillips, “Game Engines for Beginners.” [Online]. Available: https://hardcoregamer.com/x-best-game-engines-for-beginner-game-devs/

[4] R. B. Studio, “Godot vs. Unity vs. Unreal Comparison.” [Online]. Available: https://rocketbrush.com/blog/godot-vs-unity-vs-unreal-comparison

[5] CodeFriend, “How to create a multiplayer game in Unity (Setup NetCode in 10 minutes).” [Online]. Available: https://youtu.be/_NLsWFgVX6E?si=O0kLmpy46p7oked8

[6] G. Forums, “Godot vs Unity vs Unreal for Multiplayer networking?.” [Online]. Available: https://godotforums.org/d/26408-godot-vs-unity-vs-unreal-for-multiplayer-networking

[7] G. Documentation, “High-level multiplayer.” [Online]. Available: https://docs.godotengine.org/en/stable/tutorials/networking/high_level_multiplayer.html#initializing-the-network

[8] E. Games, “Multiplayer Programming Quick Start.” [Online]. Available: https://dev.epicgames.com/documentation/en-us/unreal-engine/multiplayer-programming-quick-start-for-unreal-engine